

GraphHunter

Algorithms Reference

GraphHunter Engineering

April 23, 2026

A specification-style reference for the three numerical and graph routines at the core of GraphHunter: temporal depth-first search, five-component path anomaly scoring, and SIMD-accelerated sorted set intersection.

Contents

1 Preliminaries	1
2 Temporal Depth-First Search	2
2.1 Problem	2
2.2 Algorithm	2
2.3 Complexity	3
2.4 Invariants	4
2.5 References	4
3 Path Anomaly Scoring	5
3.1 Problem	5
3.2 Components	5
3.2.1 Entity rarity	5
3.2.2 Edge rarity	5
3.2.3 Neighborhood concentration	6
3.2.4 Temporal novelty	6
3.2.5 GNN threat	6
3.3 Composite	6
3.4 Invariants	6
3.5 Trade-offs	7
3.6 References	7
4 SIMD-Accelerated Sorted Set Intersection	8
4.1 Problem	8
4.2 Dispatch	8
4.3 Scalar variant	9
4.4 AVX2 variant	9
4.5 Complexity	9
4.6 Invariants	10
4.7 References	10
Bibliography	12
Bibliography	13

Chapter 1

Preliminaries

This document specifies three algorithms used in GraphHunter’s core detection pipeline. Each chapter follows the same structure: a **problem statement** formalizing the input and output, an **algorithm** description in pseudocode, a **complexity** analysis, a list of **invariants** the implementation must satisfy, and **references** to the literature from which the techniques were drawn.

The documents are *specifications*, not code walkthroughs. Line numbers and function names drift as the codebase evolves; the math and the invariants do not. Each invariant is pinned by at least one property-based test in the Rust test suite; those tests are identified by their file name inside `core/graph-engine/tests/` or `core/matcher-ffi/tests/`.

Notation

Throughout, $G = (V, E)$ denotes a directed, labeled, time-stamped graph. Each node $v \in V$ carries an entity type $\tau(v)$ drawn from a finite alphabet. Each edge $e = (u, v, r, t) \in E$ carries a source u , destination v , relation-type label r , and an integer timestamp $t \in \mathbb{Z}$.

A *hypothesis* $H = (s_1, s_2, \dots, s_k)$ is an ordered sequence of *steps*; each step $s_i = (\tau_i^o, r_i, \tau_i^d, P_i)$ specifies an origin type, a relation type, a destination type, and a set of predicates on edge and destination metadata.

A *path* $\pi = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ is a sequence of $k + 1$ nodes connected by k edges; path indexing is zero-based.

Chapter 2

Temporal Depth-First Search

2.1 Problem

Problem 2.1 (Hypothesis matching with temporal causality). Given a graph G , a validated hypothesis $H = (s_1, \dots, s_k)$, an optional time window $[t_{\text{start}}, t_{\text{end}}]$, and a result cap $c \in \mathbb{N}$, enumerate up to c paths $\pi = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ satisfying:

1. **Type chain.** For every $i \in [1, k]$, $\tau(v_{i-1}) = \tau_i^o$ and $\tau(v_i) = \tau_i^d$. A wildcard type (ANY) matches all.
2. **Relation-type match.** There exists an edge $(v_{i-1}, v_i, r, t_i) \in E$ with $r = r_i$, or $r_i = \text{ANY}$.
3. **Temporal causality.** $t_1 \leq t_2 \leq \dots \leq t_k$.
4. **Time-window bound.** If the window is set, every $t_i \in [t_{\text{start}}, t_{\text{end}}]$.
5. **k -simplicity.** The nodes $v_0, v_1, \dots, v_k$ are pairwise distinct.
6. **Predicate match.** The edge and destination metadata satisfy every predicate in P_i .

2.2 Algorithm

The matcher performs depth-first search with backtracking, parameterized by the current hypothesis step. The pseudocode below extends the standard DFS (Cormen *et al.* [1] §22.3) with a pattern pointer, a temporal guard, and predicate evaluation.

Algorithm 1: SEARCH-TEMPORAL-PATTERN($G, H, [t_{lo}, t_{hi}], c$)

INPUT : graph G ; hypothesis H ; window $[t_{lo}, t_{hi}]$; cap c
 OUTPUT : set R of matching paths, with $|R| \leq c$; truncation flag

```

1  $R \leftarrow \emptyset$ ;  $trunc \leftarrow \perp$ 
2  $S \leftarrow \{v \in V : \tau(v) = \tau_1^o \vee \tau_1^o = \text{ANY}\}$ 
3 FOR each  $v \in S$  DO
4   IF  $|R| \geq c$  THEN
5      $trunc \leftarrow \top$ ; break
6   DFS( $v, 1, [v], \{v\}$ )
7 RETURN ( $R, trunc$ )

8 PROCEDURE DFS( $u, i, \pi, V_{\text{seen}}$ )
9   IF  $i > k$  THEN
10     $R \leftarrow R \cup \{\pi\}$ 
11    RETURN
12   FOR each edge  $(u, w, r, t) \in E$  in ascending  $t$  order DO
13     IF  $\tau(w) \neq \tau_i^d \wedge \tau_i^d \neq \text{ANY}$  THEN
14       continue
15     IF  $r \neq r_i \wedge r_i \neq \text{ANY}$  THEN
16       continue
17     IF  $w \in V_{\text{seen}}$  THEN
18       continue ▷  $k$ -simplicity
19     IF  $\pi$  non-empty and  $t < t_{\text{last}}$  THEN
20       continue
21     IF window set and  $t \notin [t_{lo}, t_{hi}]$  THEN
22       continue
23     IF predicates  $P_i$  reject  $(e, w)$  THEN
24       continue
25     DFS( $w, i + 1, \pi \cdot w, V_{\text{seen}} \cup \{w\}$ )
  
```

The driver selects candidate start nodes by type, then launches DFS from each. The procedure emits on success ($i > k$), otherwise recurses on every edge that satisfies the combined type, temporal, and predicate constraints. Backtracking unwinds the *visited* set and the partial path in LIFO order.

2.3 Complexity

Let $n = |V|$, $m = |E|$, $k = |H|$, and $b = \max_{v \in V} \deg_{\text{out}}(v)$.

Theorem 2.2 (Worst-case running time). *SEARCH-TEMPORAL-PATTERN runs in $\mathcal{O}(n \cdot b^k)$ time and uses $\mathcal{O}(k)$ stack depth plus $\mathcal{O}(n)$ memory for the visited set.*

Sketch. The outer loop over candidate start nodes is $\mathcal{O}(n)$. Each DFS call at depth i iterates at most b edges and recurses at most b times at depth $i + 1$. Unfolding the recursion to depth k yields b^k work per start node. The stack depth is bounded by k , and *visited* has $\mathcal{O}(n)$ entries. □

In practice, the type filter and the causality constraint reduce the effective branching factor to $b' \ll b$. On the **User-Auth-IP** benchmark with $n = 10,000$, observed wall time is ≈ 15 ms, consistent with $b' \cdot k \approx 30$ edge visits per start node.

Remark 2.3 (Early termination). The enumeration stops once $|R| = c$, at which point the caller receives $trunc = \top$. This caps wall time at the cost of result completeness; downstream code must treat the result set as a sample, not a census.

2.4 Invariants

The following invariants must hold for every call; violations are bugs, not edge cases.

Invariant 2.4 (Hypothesis well-formedness). *H has passed `HYPOTHESIS.VALIDATE` before `SEARCH-TEMPORAL-PATTERN` is invoked. In particular, $k \geq 1$ and $\tau_i^d = \tau_{i+1}^o$ for every $i \in [1, k - 1]$.*

Invariant 2.5 (Visited-set well-balance). *Every insertion into V_{seen} is matched by a deletion along the backtracking path. In particular, on return to the driver loop, $V_{\text{seen}} = \{v\}$ where v is the current start.*

Invariant 2.6 (Path length and acyclicity). *Every emitted $\pi \in R$ has $|\pi| = k + 1$, and its nodes are pairwise distinct.*

Implementation note. The invariants of Chapter 2 are tested in `core/graph-engine/tests/parsers.rs` and `core/matcher-ffi/tests/simd_matcher_test.rs`. The equivalence between this DFS matcher and the SIMD implementation of Chapter 4 is pinned by `core/matcher-ffi/tests/equivalence_proptest.rs`.

2.5 References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed., §22.3. MIT Press, 2009. [1]
- R. Tarjan. “Depth-first search and linear graph algorithms.” *SIAM J. Comput.*, 1(2), 1972.
- W. Fan, J. Li, S. Ma, N. Tang, Y. Wu, Y. Wu. “Graph Pattern Matching: From Intractable to Polynomial Time.” *VLDB*, 2010.

Chapter 3

Path Anomaly Scoring

3.1 Problem

Problem 3.1 (Weighted five-component anomaly score). Given a finalized graph G augmented with observation counts, a path $\pi = v_0 \rightarrow \dots \rightarrow v_k$ produced by Algorithm 1, and a non-negative weight vector $w = (w_1, w_2, w_3, w_4, w_5)$, compute a composite anomaly score $S(\pi) \in [0, 1]$ and a breakdown $C(\pi) = (C_1, C_2, C_3, C_4, C_5) \in [0, 1]^5$ of the five contributing components.

3.2 Components

Every component is bounded in $[0, 1]$ and averaged over the appropriate part of the path.

3.2.1 Entity rarity

For a node v with observation count $\text{freq}(v)$ and $\text{freq}_{\max} = \max_v \text{freq}(v)$,

$$\text{ER}(v) = \max\left(0, 1 - \frac{\ln \text{freq}(v)}{\ln \text{freq}_{\max}}\right),$$

with $\text{ER}(v) = 0$ when $\text{freq}_{\max} = 1$ (degenerate case). Averaged over the path:

$$C_1(\pi) = \frac{1}{|\pi|} \sum_{v \in \pi} \text{ER}(v).$$

The logarithmic normalization flattens the heavy-tailed observation distributions we observe in production so that a single very-popular node does not crush the score dynamic range.

3.2.2 Edge rarity

For a directed pair (s, d) with co-occurrence count $\text{freq}(s, d)$ and $\text{freq}_{\max}^{\text{pair}}$ the global maximum,

$$\text{EdgeR}(s, d) = \max\left(0, 1 - \frac{\ln \text{freq}(s, d)}{\ln \text{freq}_{\max}^{\text{pair}}}\right),$$

averaged over the $k = |\pi| - 1$ consecutive pairs in π .

3.2.3 Neighborhood concentration

Precomputed during $\text{FINALIZE}(G)$ and cached per node. The scalar $\text{NC}(v) \in [0, 1]$ encodes the classical “dense anomalous subgraph” heuristic of Akoglu *et al.* [2]: nodes with few neighbors dominated by a single type score high; nodes embedded in diverse, populous neighborhoods score low. Averaged over the path.

3.2.4 Temporal novelty

Given the first-seen timestamp $\tau_{\text{first}}(v)$ and the graph-wide extrema $\tau_{\text{min}}, \tau_{\text{max}}$,

$$\text{TN}(v) = \frac{\tau_{\text{first}}(v) - \tau_{\text{min}}}{\tau_{\text{max}} - \tau_{\text{min}}},$$

with $\text{TN}(v) = 0$ when $\tau_{\text{max}} = \tau_{\text{min}}$ (all nodes seen at the same timestamp). Averaged over the path.

3.2.5 GNN threat

If a graph neural network has produced a threat-probability estimate $\text{GNN}(v) \in [0, 1]$ per node, the fifth component is the path average of those probabilities. When the GNN cache is empty (no inference has run) the component is zero by convention, and the weight renormalization in §3.3 absorbs the deficit by redistributing mass across the remaining components.

3.3 Composite

The composite score is the weight-normalized convex combination

$$S(\pi) = \frac{\sum_{i=1}^5 w_i \cdot C_i(\pi)}{\sum_{i=1}^5 w_i}, \quad (3.1)$$

clamped to $[0, 1]$. When $\sum_i w_i = 0$ the quotient would be undefined; the implementation returns $S(\pi) = 0$ explicitly in that branch. This is the invariant that prevents **NaN** from propagating into downstream sort comparators, where it would violate strict-weak ordering.

3.4 Invariants

Invariant 3.2 (Boundedness). *For every π and every non-negative w , $S(\pi) \in [0, 1]$ and $C_i(\pi) \in [0, 1]$ for $i \in \{1, \dots, 5\}$. No component ever returns **NaN** or $\pm\infty$.*

Invariant 3.3 (Zero-weights guard). *If $\sum_i w_i = 0$, then $S(\pi) = 0$.*

Invariant 3.4 (Un-finalized scorer). *If $\text{FINALIZE}(G)$ has not been called, SCORE-PATH returns $(0, 0)$ irrespective of π . Callers use IS-FINALIZED as a gate.*

Theorem 3.5 (Entity-rarity monotonicity). *Let $a, b \in V$ satisfy $\text{freq}(a) \leq \text{freq}(b)$ and $\tau_{\text{first}}(a) = \tau_{\text{first}}(b)$. Then $\text{ER}(a) \geq \text{ER}(b)$.*

Proof. $\text{ER}(v) = 1 - \ln \text{freq}(v) / \ln \text{freq}_{\max}$ is non-increasing in $\text{freq}(v)$ because $\ln$ is non-decreasing and $\ln \text{freq}_{\max} > 0$ whenever $\text{freq}_{\max} > 1$. The $\max(0, \cdot)$ clamp preserves the ordering. $\square$

Implementation note. Invariants 3.2–3.4 and Theorem 3.5 are pinned by the property tests in `core/graph-engine/tests/anomaly_proptest.rs`, running 256 cases per property across randomized weight vectors and observation distributions.

3.5 Trade-offs

- **Averaging masks outliers.** A single extremely-rare node on a long path gets averaged down by its common neighbors. Alternatives (max-over-path, quantile) break the monotonicity property and complicate paginated consumers.
- **Log normalization assumes Zipfian distributions.** If observation counts are roughly uniform, C_1 and C_2 collapse to near zero.
- **First-seen is spoofable.** An attacker aware of the scoring can backdate $\tau_{\text{first}}(v)$ on a fresh node to suppress C_4 . Mitigation is out of scope; ingestion must cross-check against an append-only audit log.

3.6 References

- L. Akoglu, H. Tong, D. Koutra. “Graph-based anomaly detection and description: a survey.” *DMKD* 29(3), 2015. [2]
- V. Chandola, A. Banerjee, V. Kumar. “Anomaly detection: a survey.” *ACM Computing Surveys* 41(3), 2009.
- L. Akoglu, M. McGlohon, C. Faloutsos. “OddBall: Spotting Anomalies in Weighted Graphs.” *PAKDD*, 2010.

Chapter 4

SIMD-Accelerated Sorted Set Intersection

4.1 Problem

Problem 4.1 (Sorted set intersection). Given two strictly ascending arrays $A[0..m]$ and $B[0..n]$ of 32-bit unsigned integers, compute either

- $\text{INTERSECT}(A, B, out)$: write the sorted intersection $A \cap B$ into out and return $|A \cap B|$; or
- $\text{INTERSECT-COUNT}(A, B)$: return $|A \cap B|$ without materializing the output (used when the caller only needs the count for pruning).

A temporal variant $\text{INTERSECT-TEMPORAL}$ additionally filters matches whose associated timestamp lies outside a window $[t_{lo}, t_{hi}]$. Used by the matcher of Chapter 2 when narrowing candidate edges.

4.2 Dispatch

A static feature-detection routine selects one of four variants at first call and memoizes the choice in an atomic global:

ISA	Variant	Lane width	Intrinsic family
AVX-512	scalar fallback (for now)	—	—
AVX2	INTERSECT-AVX2	$8 \times \text{uint32}$	<code>_mm256_cmpeq_epi32</code> , <code>_mm256_movemask_epi8</code>
SSE4.2	scalar fallback	—	—
NEON	scalar fallback	—	—
Other	INTERSECT-SCALAR	1	—

Branch prediction eliminates the dispatch cost after the first few invocations.

4.3 Scalar variant

The two-pointer merge, for reference.

Algorithm 2: INTERSECT-SCALAR(A, B, out)

INPUT : sorted arrays $A[0..m)$, $B[0..n)$
 OUTPUT : count $k = |A \cap B|$; $out[0..k)$ holds the intersection

```

1  $i \leftarrow 0$ ;  $j \leftarrow 0$ ;  $k \leftarrow 0$ 
2 WHILE  $i < m$  AND  $j < n$  DO
3   IF  $A[i] = B[j]$  THEN  $out[k] \leftarrow A[i]$ ;  $k \leftarrow k + 1$ ;  $i \leftarrow i + 1$ ;  $j \leftarrow j + 1$ 
4   ELSE IF  $A[i] < B[j]$  THEN  $i \leftarrow i + 1$ 
5   ELSE  $j \leftarrow j + 1$ 
6 RETURN  $k$ 
```

Running time is $\Theta(m + n)$ in both the best and worst cases.

4.4 AVX2 variant

The AVX2 specialization follows the broadcast–compare–movemask scheme of Schlegel, Gubichev, and Neumann [3]. Let $V = 8$ be the AVX2 lane count.

Algorithm 3: INTERSECT-AVX2(A, B, out)

INPUT : sorted arrays $A[0..m)$, $B[0..n)$, with strictly increasing values
 OUTPUT : count k ; $out[0..k)$ holds the intersection

```

1  $i \leftarrow 0$ ;  $j \leftarrow 0$ ;  $k \leftarrow 0$ 
2 WHILE  $i + V \leq m$  AND  $j < n$  DO
3    $\mathbf{v}_A \leftarrow \text{LOAD-256}(A + i)$  ▷ 8 lanes of  $A$ 
4    $a_{\max} \leftarrow A[i + V - 1]$ 
5   WHILE  $j < n$  AND  $B[j] \leq a_{\max}$  DO
6      $\mathbf{v}_B \leftarrow \text{BROADCAST-256}(B[j])$ 
7      $\mathbf{c} \leftarrow \mathbf{v}_A \stackrel{?}{=} \mathbf{v}_B$  ▷ lane-wise equality
8      $M \leftarrow \text{MOVEMASK}(\mathbf{c})$ 
9     IF  $M \neq 0$  THEN  $out[k] \leftarrow B[j]$ ;  $k \leftarrow k + 1$ 
10     $j \leftarrow j + 1$ 
11   $i \leftarrow i + V$ 
12 scalar tail for  $i \geq m - V + 1$ 
13 RETURN  $k$ 
```

Because both arrays are strictly ascending, at most one lane of $\mathbf{c}$ can be nonzero per broadcast; the MOVEMASK result therefore encodes a simple “hit or miss.” The scalar tail ensures no out-of-bounds load when $m \not\equiv 0 \pmod{V}$.

4.5 Complexity

Let m, n be the input sizes and $k = |A \cap B|$ the output size.

Theorem 4.2 (AVX2 asymptotics). *INTERSECT-AVX2 runs in $\mathcal{O}(m + n)$ time. Empirically, the constant factor is 3–4 $\times$ smaller than INTERSECT-SCALAR on uniform-density inputs, and up to 6–8 $\times$ smaller when one array is sparse relative to the other (the inner B-loop executes multiple broadcasts per outer A-step).*

Remark 4.3 (Pathological case). When every $B[j] < \max(\mathbf{v}_A)$ for nearly every outer step, the inner loop can traverse the full length of B per outer iteration, degenerating toward $\mathcal{O}(m \cdot n)$. The matcher sidesteps this by invoking intersection with the smaller array as the outer loop.

4.6 Invariants

Invariant 4.4 (Input well-formedness). *A and B are each strictly ascending with no duplicates. Violation causes silent duplicates in the output; the matcher’s adjacency-list builder guarantees this property upstream.*

Invariant 4.5 (Output capacity). *The caller allocates out with room for at least $\min(m, n)$ elements.*

Invariant 4.6 (No SIMD overread). *The vectorized loop processes only full V-lane vectors; the tail falls back to scalar. No unaligned `loadu` reads past the end of A or B, so the algorithm never takes a page fault beyond the allocated regions.*

Invariant 4.7 (Thread-safety). *The ISA selection is written once behind an atomic flag; the specialized variants are pure functions. Safe to call concurrently from multiple matcher threads.*

Invariant 4.8 (ABI stability). *The Rust bridge asserts `gm_abi_version() == EXPECTED_ABI_VERSION` at first call via `std::sync::Once`. Silent drift in the dispatch table layout therefore fails loudly at initialization rather than at runtime with memory corruption.*

Implementation note. The full SIMD matcher is tested end-to-end (not intersection alone) by `core/matcher-ffi/tests/equivalence_proptest.rs`, which enumerates random small graphs and hypotheses and requires the DFS and SIMD result sets to be identical. A bug in INTERSECT-AVX2 surfaces there as a missing path.

4.7 References

- B. Schlegel, A. Gubichev, T. Neumann. “Fast Sorted-Set Intersection using SIMD Instructions.” *ADMS*, 2011. [3]
- D. Lemire, O. Kurz, C. Rupp. “Fast random integer generation in an interval.” *ACM TOMS* 45(1), 2019.

-
- *Intel 64 and IA-32 Architectures Software Developer's Manual*, Vol. 2B, §VPCMPEQD, VPMOVMASKB.

Bibliography

Bibliography

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [2] L. Akoglu, H. Tong, D. Koutra. “Graph-based anomaly detection and description: a survey.” *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015.
- [3] B. Schlegel, A. Gubichev, T. Neumann. “Fast Sorted-Set Intersection using SIMD Instructions.” *International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures (ADMS)*, 2011.